

PANADERO Baking System

Professor-Style Defense Questions and Answers

English + Tagalog Reviewer

This reviewer prepares the developer for a project defense, consultation, or oral examination about the local PANADERO bakery ordering system.

Recommended answering pattern:

1. Give the direct answer.
 2. Explain how it is implemented in PANADERO.
 3. Mention why the design is useful.
-

Part 1 — General system questions

1. What is the PANADERO system?

English answer:

PANADERO is a local bakery ordering and management system. It has customer, staff, and admin portals. Customers can browse products, add items to a cart, place orders, make reservations, and track their requests. Staff can manage inventory and request statuses. Admins can manage catalog records, users, stock, and system activity.

Tagalog explanation:

Ang PANADERO ay isang local bakery ordering at management system. May tatlong portal: customer, staff, at admin. Ang customer ay maaaring tumingin ng products, mag-cart, umorder, magpa-reserve, at mag-track ng order. Ang staff ay nag-aasikaso ng inventory at status ng orders. Ang admin naman ay namamahala sa products, users, stock, at system records.

2. What problem does the system solve?

English answer:

It organizes bakery operations that would otherwise be handled manually. It gives customers a structured way to order, gives staff a way to monitor stock and requests, and gives administrators a central view of products, users, orders, reservations, and inventory.

Tagalog explanation:

Inaayos ng system ang mga proseso na maaaring mano-manong ginagawa. Mas madali para sa customer ang pag-order, mas madaling makita ng staff ang stock at requests, at may central record ang admin para sa products, users, orders, reservations, at inventory.

3. Why did you separate the system into frontend, backend, and database?

English answer:

The separation follows a layered architecture. The frontend is responsible for presentation and user interaction. The backend is responsible for validation, authentication, business rules, and communication with MySQL. The database is responsible for permanent storage. This separation makes the system easier to maintain and prevents browser code from connecting directly to the database.

Tagalog explanation:

Hinati ang system sa layers para malinaw ang trabaho ng bawat bahagi. Ang frontend ang nagpapakita ng interface. Ang backend ang nagva-validate at nag-aapply ng rules. Ang database ang permanenteng nag-iimbak ng data. Mas safe ito dahil hindi direktang kumokonekta ang browser sa MySQL.

4. What happens when a user opens the system?

English answer:

The user runs the one-click launcher or starts the server manually. Node.js starts Express on port 3000. The server connects to MySQL, runs its startup migration and seed checks, and serves the login page. The browser then loads the HTML, CSS, and JavaScript files. The JavaScript calls the API when it needs data.

Tagalog explanation:

Kapag binuksan ang system, nagsisimula ang Node.js server sa port 3000. Kumokonekta ito sa MySQL at chine-check kung mayroon ang required tables at initial data. Pagkatapos, ise-serve nito ang login page at frontend files. Kapag kailangan ng data, ang JavaScript ay tatawag sa API.

5. Why is the browser not allowed to connect directly to MySQL?

English answer:

A browser should never receive database credentials. Direct access would expose the database to users and would allow them to bypass validation, permissions, prices, and stock rules. The backend acts as a controlled gateway between the browser and MySQL.

Tagalog explanation:

Hindi dapat direktang kumonekta ang browser sa MySQL dahil maaaring makita o ma-expose ang database credentials. Maaari ring ma-bypass ang validation, permissions, presyo, at stock rules. Ang Node.js backend ang nagsisilbing secure na tagapamagitan.

6. What is the role of HTML, CSS, and JavaScript?

English answer:

HTML defines the page structure and form controls. CSS defines the visual design and layout. JavaScript handles user events, communicates with the API, renders returned records, manages the temporary cart, and updates the interface.

Tagalog explanation:

Ang HTML ang structure ng page. Ang CSS ang design at layout. Ang JavaScript ang naghandle ng clicks, forms, API calls, display ng data, at temporary cart.

7. What is the role of Node.js and Express?

English answer:

Node.js runs the backend JavaScript outside the browser. Express provides the web server and API routing. In PANADERO, Express receives HTTP requests, checks authentication and roles, validates data, executes MySQL queries, and returns JSON responses.

Tagalog explanation:

Ang Node.js ang nagpapatakbo ng backend JavaScript. Ang Express ang nagse-set up ng web server at routes. Tumatanggap ito ng requests, nagche-check ng login at role, nagva-validate ng data, kumokonekta sa MySQL, at nagbabalik ng JSON response.

Part 2 — Frontend and API questions

8. How does the customer menu get products from SQL?

English answer:

`menu.js` calls `ProductService.refreshProducts()`. The service sends `GET /api/catalog`. The backend queries `categories` and `bread_products`, maps the database rows into frontend product objects, and returns JSON. `menu.js` then renders the product cards.

Tagalog explanation:

Ang `menu.js` ay tumatawag sa `ProductService`. Ang service ay gumagamit ng `GET /api/catalog`. Ang backend ay kumukuha ng records mula sa `categories` at `bread_products`, ginagawa itong JSON, at ipinapakita ng `menu.js` bilang product cards.

9. What is an API endpoint?

English answer:

An API endpoint is a specific URL and HTTP method used to request or change data. For example, `GET /api/catalog` reads products, while `POST /api/orders` creates an order.

Tagalog explanation:

Ang API endpoint ay isang specific URL at method para kumuha o magbago ng data. Halimbawa, ang `GET /api/catalog` ay kumukuha ng products, at ang `POST /api/orders` ay gumagawa ng order.

10. What is the difference between GET, POST, and PATCH in this system?

English answer:

- `GET` reads data without changing it.
- `POST` creates a new record or performs a create action.
- `PATCH` changes part of an existing record, such as an order status or stock value.

Examples are `GET /api/catalog`, `POST /api/orders`, and `PATCH /api/orders/:id/status`.

Tagalog explanation:

- GET ay kumukuha ng data.
 - POST ay gumagawa ng bagong record o action.
 - PATCH ay nagbabago ng isang existing record, tulad ng status o stock.
-

11. Why does the frontend send product IDs and quantities instead of the total price?

English answer:

The total price sent by a browser cannot be trusted because a user can modify browser data. The backend reads the current product prices from MySQL and calculates the total itself. This prevents price manipulation.

Tagalog explanation:

Hindi pinagkakatiwalaan ang total price na galing sa browser dahil maaaring baguhin ito ng user. Product ID at quantity lang ang ipinapadala. Ang backend ang kumukuha ng totoong presyo sa MySQL at nagko-compute ng final total.

12. What is stored in the browser and what is stored in MySQL?

English answer:

The temporary cart and basic login token are stored in browser local storage. Permanent business records are stored in MySQL: users, categories, products, stock, orders, order items, reservations, and inventory batches.

Tagalog explanation:

Ang temporary cart at login token ay nasa browser local storage. Ang permanent records tulad ng users, products, stock, orders, reservations, at batches ay nasa MySQL.

13. Why is the cart stored temporarily in the browser?

English answer:

The cart is only a draft of what the customer intends to buy. It becomes a permanent business record only after checkout succeeds. This keeps the database from filling with abandoned carts.

Tagalog explanation:

Ang cart ay parang draft lamang ng gustong bilhin ng customer. Nagiging permanent record lang ito kapag successful ang checkout. Kaya hindi napupuno ang database ng mga abandoned cart.

14. Why can the design change without changing the database?

English answer:

The design is separated from the service layer. As long as the HTML keeps the expected IDs, form names, and script contracts, the CSS and layout can change while the JavaScript continues to call the same API.

Tagalog explanation:

Hiwalay ang design sa data services. Maaaring baguhin ang kulay, layout, at images basta hindi sinisira ang important IDs, form names, at JavaScript functions na ginagamit ng API connection.

Part 3 — Authentication and security questions

15. How does login work?

English answer:

The user submits an email and password. The frontend sends them to `POST /api/auth/login`. The server finds the user by email, compares the submitted password with the bcrypt hash, creates a temporary session token, and returns the user role. The login page redirects the user to the appropriate portal.

Tagalog explanation:

Ipinapadala ang email at password sa backend. Hahanapin ng server ang email sa `users`, iko-compare ang password sa bcrypt hash, gagawa ng session token, at titingnan ang role para ma-redirect sa tamang portal.

16. Why do you hash passwords?

English answer:

Passwords must not be stored as readable text. Bcrypt converts a password into a one-way hash. During login, the submitted password is compared with the hash without storing the original password.

Tagalog explanation:

Hindi dapat plain text ang password sa database. Ginagawang one-way hash ito gamit ang bcrypt. Sa login, chine-check lang kung tugma ang password sa hash; hindi kailangang malaman ng system ang original password.

17. What is a Bearer token?

English answer:

A Bearer token is a temporary credential sent in the HTTP `Authorization` header. The backend uses it to identify the logged-in user for protected requests.

```
Authorization: Bearer <token>
```

Tagalog explanation:

Ang Bearer token ay temporary proof na naka-login ang user. Ipinapadala ito sa `Authorization` header para makilala ng backend kung sinong user ang gumagawa ng request.

18. How does the system enforce roles?

English answer:

The backend uses middleware. `auth` checks whether the token identifies a session. `staff` allows staff or admin users. `admin` allows only admin users. Customer-only endpoints verify that the logged-in role is `customer`.

Tagalog explanation:

May middleware na nagche-check ng roles. Ang `auth` ay nagche-check ng valid token. Ang `staff` ay para sa staff at admin. Ang `admin` ay para sa admin lang. Ang orders at reservations ng customer ay nangangailangan ng customer role.

19. What happens if a customer tries to access an admin endpoint?

English answer:

The backend rejects the request with HTTP 403 and an `Admin access required` message. The role check happens on the server, so hiding a button in the frontend is not the only protection.

Tagalog explanation:

Ibabalik ng backend ang 403 error dahil hindi admin ang user. Mahalaga na nasa server ang role check; hindi sapat na itago lang ang admin button sa frontend.

20. What is SQL injection and how does PANADERO reduce the risk?

English answer:

SQL injection happens when untrusted input is inserted into SQL syntax. PANADERO uses parameterized queries such as `WHERE email=?` and supplies values separately. This prevents input from being interpreted as SQL code.

Tagalog explanation:

Ang SQL injection ay kapag ang input ng user ay nagagamit bilang SQL command. Gumagamit ang PANADERO ng parameterized queries at placeholders tulad ng `?`, kaya hiwalay ang data sa SQL syntax.

21. Why should `.env` not be shared?

English answer:

`.env` contains database connection settings and may contain the MySQL password. It is ignored by Git and should remain local.

Tagalog explanation:

Maaaring naglalaman ang `.env` ng MySQL password. Kaya hindi ito dapat i-upload sa GitHub o ibigay sa ibang tao. Ang bawat laptop ay dapat may sariling `.env`.

22. Is the current local authentication production-ready?

English answer:

It is suitable for local testing, but it is not fully production-ready. The session map is stored in Node.js memory, so restarting the server logs users out. A production system should use persistent sessions or secure JWT handling, HTTPS, rate limiting, CSRF protection, and stronger audit logging.

Tagalog explanation:

Para sa local testing ay sapat ito, pero hindi pa ideal para sa public production. Nasa memory lang ng Node.js ang sessions, kaya mawawala ang login kapag ni-restart ang server. Para sa production, kailangan ng persistent sessions, HTTPS, rate limiting, CSRF protection, at mas maayos na audit logs.

Part 4 — SQL and database questions

23. What is the name of the database?

English answer:

The database is named `panadero_bakery`. The SQL file creates it if it does not exist and selects it with the `USE panadero_bakery` statement.

Tagalog explanation:

Ang pangalan ng database ay `panadero_bakery`. Awtomatikong ginagawa ito ng SQL file kung wala pa, at ginagamit ito sa pamamagitan ng `USE panadero_bakery`.

24. What tables are in the system?

English answer:

The main tables are `users`, `categories`, `bread_products`, `orders`, `order_items`, `reservations`, `inventory_batches`, and `system_logs`.

Tagalog explanation:

Ang pangunahing tables ay `users`, `categories`, `bread_products`, `orders`, `order_items`, `reservations`, `inventory_batches`, at `system_logs`.

25. What is the purpose of the `users` table?

English answer:

It stores account identity and access data: user ID, username, email, password hash, role, and creation timestamp.

Tagalog explanation:

Nagtatago ito ng impormasyon para sa account at access: user ID, pangalan, email, hashed password, role, at petsa ng paggawa ng account.

26. Why is email unique?

English answer:

Email is used as the login identifier. A unique constraint prevents two accounts from using the same email and avoids ambiguous login behavior.

Tagalog explanation:

Ginagamit ang email sa login. Kailangan itong unique para walang dalawang account na parehong email at para malinaw kung aling account ang ila-log in.

27. What is the purpose of the `categories` table?

English answer:

It prevents category text from being repeated inconsistently in every product row. Products point to a category using `category_id`.

Tagalog explanation:

Pinaghihiwalay nito ang category records para hindi paulit-ulit at iba-iba ang spelling sa bawat product. Ang product ay tumutukoy sa category gamit ang `category_id`.

28. Why is `bread_products` named that way even though it stores cakes, doughnuts, and pies?

English answer:

The table name came from the supplied database schema. Its actual responsibility is the bakery product catalog, not only bread. Renaming it would require migration work and changes throughout the backend, so the existing compatible name was retained.

Tagalog explanation:

Galing sa original SQL schema ang pangalan na `bread_products`. Kahit ganoon ang pangalan, lahat ng bakery products ang laman nito. Hindi ito pinalitan para manatiling

compatible ang existing system at queries.

29. What is a primary key?

English answer:

A primary key uniquely identifies each row. Examples are `user_id` , `category_id` , `product_id` , and `order_id` .

Tagalog explanation:

Ang primary key ay unique identifier ng bawat row. Halimbawa, ang `order_id` ang unique number ng bawat order.

30. What is a foreign key?

English answer:

A foreign key connects a row to a related row in another table. For example, `orders.user_id` points to `users.user_id` , and `order_items.order_id` points to `orders.order_id` .

Tagalog explanation:

Ang foreign key ang nagli-link sa related tables. Halimbawa, ang `orders.user_id` ay tumutukoy sa customer sa `users` table.

31. What does one-to-many mean in the ERD?

English answer:

One-to-many means one record in the first table can relate to many records in the second table. One user can place many orders, but each order belongs to one user.

Tagalog explanation:

Ang one-to-many ay ibig sabihin isang record ay maaaring magkaroon ng maraming related records. Halimbawa, isang customer ay maaaring magkaroon ng maraming orders, pero bawat order ay may isang owner na user.

32. Why are orders divided into `orders` and `order_items` ?

English answer:

This is a header/detail design. `orders` stores information about the entire transaction, such as customer, total, status, and date. `order_items` stores each product line. This allows one order to contain multiple products without repeating order-level information.

Tagalog explanation:

Ang `orders` ay header ng transaction, habang ang `order_items` ay detalye ng bawat product. Kaya maaaring maraming products sa isang order nang hindi inuulit ang customer, date, at total sa bawat row.

33. Why is `unit_price` stored in `order_items` ?

English answer:

It preserves the historical price used at checkout. If the catalog price changes later, old orders still show the correct original price.

Tagalog explanation:

Para mai-save ang presyo noong mismong pag-order. Kapag nagbago ang current product price sa future, tama pa rin ang presyo ng lumang order.

34. Why is `total_amount` stored in `orders` if it can be calculated from order items?

English answer:

It provides a fast order summary and preserves the finalized transaction amount. The backend still calculates it from products and quantities during checkout before saving it.

Tagalog explanation:

Naka-save ang total para mabilis makita ang summary at para manatili ang finalized amount ng transaction. Pero hindi ito basta tinatanggap mula sa browser; backend ang nagko-compute nito.

35. What is normalization in this database?

English answer:

Normalization means organizing data to reduce unnecessary duplication and update anomalies. PANADERO separates users, categories, products, orders, and order details into related tables. For example, category information is stored once in `categories` instead of repeated as full text in every product record.

Tagalog explanation:

Ang normalization ay pag-aayos ng data para mabawasan ang duplication at inconsistency. Hiwalay ang users, categories, products, orders, at order details, kaya mas madaling i-update at mas kaunti ang paulit-ulit na data.

36. Why use **DECIMAL** for money instead of **FLOAT** ?

English answer:

DECIMAL is appropriate for currency because it stores exact decimal values. Floating-point types can introduce rounding issues.

Tagalog explanation:

Mas tama ang **DECIMAL** para sa pera dahil exact ang decimal values nito. Ang **FLOAT** ay maaaring magkaroon ng rounding errors.

37. Why use **ENUM** for statuses and roles?

English answer:

The enum restricts values to known options such as **Pending** , **Processing** , **Ready** , **Completed** , and **Cancelled** . This prevents invalid status values from entering the database.

Tagalog explanation:

Nililimitahan ng **ENUM** ang values sa valid options lamang. Pinipigilan nito ang maling status gaya ng typo o hindi suportadong status.

38. What is the purpose of **ON DELETE CASCADE** on **order_items** ?

English answer:

If an order is deleted, its dependent order items should also be deleted. The cascade keeps child records from being orphaned.

Tagalog explanation:

Kapag dinelete ang order, dapat mawala rin ang order items na kabilang dito. Pinipigilan ng cascade ang mga detail rows na maiwan nang walang parent order.

39. Why do other foreign keys use **ON DELETE RESTRICT** ?

English answer:

Restrict protects important history. A product, user, or category should not be deleted if existing records still depend on it, because that could destroy order or inventory history.

Tagalog explanation:

Pinoprotektahan ng restrict ang historical records. Hindi dapat basta mabura ang product o user kung may orders, reservations, o inventory records pa itong konektado.

40. What are the **reservations** and **inventory_batches** tables for?

English answer:

reservations stores scheduled product requests. **inventory_batches** stores each batch baked by staff, including quantity, baked date, expiration date, and notes. These tables are created by the backend migration if they are missing from the supplied SQL.

Tagalog explanation:

Ang **reservations** ay para sa scheduled product requests. Ang **inventory_batches** ay para sa bawat batch na niluto o inihanda ng staff, kasama ang quantity at expiration date. Awtomatikong ginagawa ng backend ang tables kung wala pa.

Part 5 — Order, stock, and transaction questions

41. How does the system prevent ordering more than the available stock?

English answer:

The backend reads the product row from MySQL, checks **stock_quantity**, validates the requested quantity, and rejects the order if stock is insufficient. The check happens on the server, not only in the browser.

Tagalog explanation:

Kumukuha ang backend ng current stock sa MySQL at chine-check kung sapat ito sa quantity na hinihingi. Kapag kulang, nire-reject ang order. Hindi lang browser ang gumagawa ng stock check.

42. Why does the order process use a database transaction?

English answer:

An order requires several related writes: order header, order items, and stock deduction. A transaction guarantees that either all succeed or all fail. This prevents partial orders and incorrect inventory.

Tagalog explanation:

Maraming writes ang isang order: order header, order items, at stock deduction. Kailangan ng transaction para sabay-sabay silang ma-save o sabay-sabay ma-cancel kapag may error.

43. What is rollback?

English answer:

Rollback cancels all database changes made during the current transaction. If an item is unavailable or an insert fails, the database returns to its previous consistent state.

Tagalog explanation:

Ang rollback ay nag-a-undo ng lahat ng changes sa transaction kapag may error. Kaya walang naiwan na half-created order o maling stock.

44. What happens when staff adds an inventory batch?

English answer:

The backend inserts a row into `inventory_batches` and increases `bread_products.stock_quantity` by the batch quantity inside one transaction.

Tagalog explanation:

Mag-iinsert muna ng batch record at tataasan ang current stock ng product. Ginagawa ito sa iisang transaction para consistent ang batch history at stock.

45. What happens if two customers order the same product at nearly the same time?

English answer:

The backend uses a transaction and locks the selected product rows with `FOR UPDATE` while checking stock. This helps serialize competing updates and reduces the chance of selling the same stock twice.

Tagalog explanation:

Gumagamit ang backend ng transaction at row locking habang chine-check ang stock. Nakakatulong ito para hindi parehong ma-approve ang dalawang order kung iisa na lang ang natitirang stock.

46. Why should stock be decreased only after validation?

English answer:

If stock were decreased before checking all items, a failed order could still remove inventory. The correct sequence is validate all products and quantities first, then insert records and deduct stock within the transaction.

Tagalog explanation:

Kung babawasan agad ang stock bago matapos ang validation, maaaring mabawasan ang inventory kahit failed ang order. Dapat ma-validate muna ang lahat bago mag-insert at mag-deduct.

47. Why does the customer see a reference number?

English answer:

A reference such as `ORD-00005` gives the customer and staff a readable identifier for tracking. The database primary key remains the numeric `order_id`, while the formatted reference is a user-friendly representation.

Tagalog explanation:

Ang reference number ay madaling gamitin ng customer at staff sa pag-track. Ang database ay may numeric `order_id`, pero ipinapakita sa user ang mas readable na format gaya ng `ORD-00005`.

Part 6 — Professor challenge questions

48. What if the frontend sends a fake price of ₱1.00?

English answer:

The backend ignores the browser's price. It reads the product price from `bread_products` and calculates the total using the database value. Therefore, the fake price does not change the saved order total.

Tagalog explanation:

Hindi tatanggapin ng backend ang fake price. Kukunin nito ang totoong presyo mula sa MySQL at iyon ang gagamitin sa total.

49. What if a customer changes their role in browser local storage?

English answer:

Changing local storage does not grant real access. Protected backend endpoints use the server-side session token and the role stored in the authenticated session. The server rejects unauthorized roles.

Tagalog explanation:

Kahit baguhin ng customer ang role sa browser, wala siyang tunay na access. Ang backend ang nagche-check ng session token at role, kaya ire-reject ang unauthorized request.

50. Why not put all products in JavaScript instead of MySQL?

English answer:

Hardcoded JavaScript data is not suitable for shared business state. It cannot reliably synchronize stock, orders, users, or admin changes. MySQL gives a central persistent source of truth.

Tagalog explanation:

Hindi sapat ang hardcoded products sa JavaScript dahil hindi nito kayang mag-share ng updated stock, orders, users, at admin changes. Ang MySQL ang central at permanent source of truth.

51. Why does the server seed products if the SQL file already exists?

English answer:

The startup seed check makes the application more portable. If the database exists but has no categories or products, the server can still prepare a usable local demo catalog. The count check prevents duplicate default records on every startup.

Tagalog explanation:

May seed check para gumana pa rin ang application kahit existing ang database pero walang catalog records. Check muna ang count para hindi paulit-ulit ang default products sa bawat start.

52. What happens if the database is empty?

English answer:

The SQL creates the base structure. Then `server.js` creates missing compatibility tables, inserts default categories and products if needed, and creates the demo admin and staff users.

Tagalog explanation:

Gagawa muna ang SQL ng base tables. Pagkatapos, ang `server.js` ang gagawa ng compatibility tables, default categories, products, at demo users kung wala pa.

53. What happens if the MySQL server is offline?

English answer:

The backend cannot complete its startup migration and API health checks. The launcher reports the problem, and API requests return a database-unavailable response. The solution is to start MySQL and run the launcher again.

Tagalog explanation:

Hindi makakapag-start nang maayos ang backend kung offline ang MySQL. Magpapakita ito ng database error. Kailangang i-start ang MySQL at patakbuhan ulit ang launcher.

54. Why is this called a local system?

English answer:

The browser, Node.js server, and MySQL database all run on the same laptop. The system uses `localhost` and does not require a hosted database or cloud deployment.

Tagalog explanation:

Local system ito dahil nasa iisang laptop ang browser, Node.js server, at MySQL database. Gumagamit ito ng `localhost` at hindi kailangan ng cloud database.

55. What is the biggest limitation of the current local system?

English answer:

The main limitation is that the session store is in Node.js memory. It is acceptable for a local prototype, but it should be replaced with a persistent session system for a multi-user production deployment. Staff and admin security should also be reviewed before public deployment.

Tagalog explanation:

Ang pinakamalaking limitation ay nasa memory lang ng Node.js ang sessions. Okay ito sa local prototype, pero kailangan ng persistent session system kung public o maraming users. Kailangan ding dagdagan ang security ng staff at admin portals.

56. How would you improve the system in the future?

English answer:

I would add persistent sessions, stronger staff/admin login protection, HTTPS, rate limiting, CSRF protection, real password reset emails, payment integration, audit logging, database backups, and automated tests. I would also consider separate address and payment tables if those features become permanent requirements.

Tagalog explanation:

Idadagdag ko ang persistent sessions, mas secure na staff/admin login, HTTPS, rate limiting, CSRF protection, totoong password reset, payment integration, audit logs, backups, at automated tests. Maaari ring gumawa ng hiwalay na address at payment tables kapag kailangan na talaga.

Part 7 — SQL defense quick reference

57. Explain this SQL statement

```
SELECT o.*, u.username, u.email
FROM orders o
JOIN users u ON u.user_id = o.user_id
WHERE o.user_id = ?
ORDER BY o.order_date DESC;
```

English answer:

It selects orders belonging to the logged-in user. The `JOIN` adds the user's name and email. The `WHERE` prevents showing another user's orders. The placeholder `?` receives the authenticated user ID safely. The results are sorted newest first.

Tagalog explanation:

Kinukuha nito ang orders ng kasalukuyang naka-login na user. Ang `JOIN` ay kumukuha ng username at email. Ang `WHERE` ay nagsisigurong sariling orders lang ang makikita. Ang `?` ay safe parameter para sa user ID.

58. Explain this foreign key

```
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE RESTRICT
```

English answer:

It requires the referenced user to exist before an order or reservation can point to that user. `ON DELETE RESTRICT` prevents deleting the user while dependent business records still exist.

Tagalog explanation:

Kailangan munang may existing user bago siya maging owner ng order o reservation. Pinipigilan ng `ON DELETE RESTRICT` na mabura ang user habang may dependent records pa.

59. Explain this order item relationship

```
FOREIGN KEY (order_id)
REFERENCES orders(order_id)
ON DELETE CASCADE
```

English answer:

Each order item must belong to an existing order. If the parent order is deleted, its detail rows are automatically deleted so orphaned items are not left behind.

Tagalog explanation:

Bawat order item ay dapat kabilang sa existing order. Kapag dinelete ang parent order, automatic ding made-delete ang related details.

60. What would happen if the database had no foreign keys?

English answer:

The database could contain orphaned records, such as order items pointing to nonexistent orders or products. Application code would have to detect every inconsistency manually. Foreign keys let MySQL enforce important relationships.

Tagalog explanation:

Maaaring magkaroon ng sirang relationships, tulad ng order item na walang existing order o product. Mas maraming manual checks ang kailangan sa code. Ang foreign keys ang tumutulong mag-enforce ng data integrity.

Part 8 — Short oral-defense answers

These are useful when the professor expects a quick answer.

Question	Short answer
What is the system?	A local bakery ordering and management system with customer, staff, and admin portals.
What is the backend?	Node.js and Express, responsible for API routes, validation, roles, and MySQL access.
What is the database?	MySQL database named <code>panadero_bakery</code> .
Why not direct browser-to-MySQL?	It would expose credentials and bypass server-side validation.
What is the primary order table?	<code>orders</code> stores the transaction header.
Where are products stored?	<code>bread_products</code> , linked to <code>categories</code> .
Where are order products stored?	<code>order_items</code> , linked to <code>orders</code> and <code>bread_products</code> .
Why use transactions?	To save the order and deduct stock atomically.
Why use bcrypt?	To store passwords as secure hashes rather than plain text.
What protects admin routes?	Authentication plus admin-role middleware.
What is the local URL?	<code>http://localhost:3000/login.html</code> .
What happens if stock is insufficient?	The backend rejects the request and rolls back the transaction.
What is the main local limitation?	Sessions are stored in Node.js memory and disappear on restart.
What does the ERD show?	The tables, keys, and relationships in the database.

Part 9 — Suggested defense opening statement

English version

PANADERO is a local bakery ordering and management system built using HTML, CSS, JavaScript, Node.js, Express, and MySQL. The frontend contains customer, staff, and admin portals. The browser communicates with the backend through JSON API endpoints, while the backend handles authentication, role authorization, server-side pricing, stock validation, transactions, and database access. MySQL stores the permanent business records, including users, products, orders, order items, reservations, and inventory batches. The system is designed for local laptop use and includes a one-click launcher that prepares the database and starts the application.

Tagalog version

Ang PANADERO ay isang local bakery ordering at management system na ginawa gamit ang HTML, CSS, JavaScript, Node.js, Express, at MySQL. May customer, staff, at admin portals ang frontend. Kumokonekta ang browser sa backend gamit ang JSON API endpoints. Ang backend ang nagha-handle ng authentication, role authorization, pricing, stock validation, transactions, at database access. Ang MySQL ang nagtatago ng permanent business records tulad ng users, products, orders, order items, reservations, at inventory batches. Para ito sa local laptop use at may one-click launcher para awtomatikong ma-prepare ang database at ma-start ang application.

Part 10 — Final reminder for the defense

When answering, do not only say **what** the system does. Explain **why** it was designed that way.

A strong answer usually contains:

What it does

- + Which file or table does it
- + Why that design is useful
- + What limitation remains

Example:

The order total is calculated by the backend using prices from MySQL. The checkout JavaScript sends product IDs and quantities, while `server.js` reads the authoritative prices and writes `orders` and `order_items` inside a transaction. This prevents price manipulation and keeps stock consistent. The current limitation is that the local session store is in memory, so production deployment would need persistent sessions.

Tagalog pattern:

Ang total ng order ay kinukuwenta ng backend gamit ang presyo mula sa MySQL. Product IDs at quantities lang ang ipinapadala ng checkout JavaScript. Ang `server.js` ang kumukuha ng totoong presyo at nagsa-save ng `orders` at `order_items` sa isang transaction. Pinipigilan nito ang price manipulation at maling stock. Ang limitation ay nasa memory lang ang local session, kaya kailangan ng persistent sessions para sa production.